

Setup. I did everything on an Ubuntu ARM64 VM running in UTM on my Mac, with Docker 29 and a 2-node minikube cluster. The app is a small FastAPI service with `POST /predict` and `GET /healthz`, running a TF-IDF + MultinomialNB model trained on the 1,000-row dataset from the brief. It gets 1.00 accuracy on the test split, which makes sense because the data comes from templates, so it's really easy to separate. The ML part isn't the point here anyway. All the numbers below come from files in `evidence/`.

Q1: Single-stage vs multi-stage

	naive	multi-stage
docker images size	2.41 GB	824 MB
Content size	630 MB	177 MB
Base image	3.11: 1.62 GB	3.11-slim: 226 MB
which gcc	/usr/bin/gcc	not there
pip cache	90 MB	none
apt lists	21 MB	244 KB

Both images give the same answers on `/predict` and `/healthz` (spam and ham for the same inputs). The multi-stage image is **65.8% smaller**.

Why it's smaller. Most of the drop is just the base image. The naive build uses the full `python:3.11` image (1.62 GB), which comes with a whole Debian setup including gcc and the other build tools. My runtime stage uses `python:3.11-slim` (226 MB) instead, and that swap alone saves 1.39 GB of the 1.59 GB. I checked with `which gcc`: the naive image has `/usr/bin/gcc` and the multi-stage one has nothing. The rest is stuff that stayed behind in the builder stage: 90 MB of pip cache (the naive build doesn't use `--no-cache-dir`), 21 MB of apt lists, and the whole build folder in `/app`, so `spam_dataset.csv`, `generate_dataset.py`, `train.py` and even the Dockerfiles. The multi-stage runtime only has `app/` and the 16 KB `model.joblib` that the builder made.

Q2: Compose with a Redis cache

Before predicting, the API hashes the text (SHA-256) and checks Redis for that key. On a miss it runs the model, saves the label in Redis with a 300 s TTL and returns it. On a hit it just returns the saved label. Every response also has an `X-Cache: HIT|MISS` header so you can actually see it happening. In `docker-compose.yml` the `api` service is built from my multi-stage Dockerfile and reaches the cache service (`redis:7-alpine`) using just the name `cache` as the hostname, and Compose's DNS handles that. The API waits until Redis passes its PING healthcheck before starting, and I didn't publish any Redis port to the host since only the API needs it. If Redis isn't there the API still works, just without caching, which is why the same image also works for Q1.

200 identical-text pairs	MISS	HIT	speedup
Server-side work (mean)	0.210 ms	0.135 ms	1.56×
Round trip incl. HTTP (mean)	1.406 ms	1.077 ms	1.31×

The script checks that every hit returns the same label as its miss. The time saved is small in absolute terms because this model is really fast (well under a millisecond), but it's consistent over all 400 requests, and with a heavier model the saving would be much bigger.

Compose vs Kubernetes. One container is basically one process with one lifecycle. If I want Redis too, I'd either have to put both in the same image, so restarting Redis kills the API as well, or start and connect two containers by hand every time and deal with the network, hostnames, env vars and start order myself. Compose puts all of that in one file: both

services, a network where `cache` works as a hostname, and the startup dependency. But Compose only works on one machine. It has no scheduler, no idea of nodes, doesn't keep replicas alive and can't do rolling updates. Kubernetes is the cluster version of this. It spreads containers over many machines, keeps the replica count up and rolls out new versions without downtime, which is exactly what Q3 and Q4 test.

Q3: Indexed Job for parallel validation

This one is a separate batch job. I made 8 shards of 250 signup records each with a seeded script, and every shard has a known number of bad rows (a blank required field or a broken email). The expected counts are saved in `expected_invalid.json` so I can check the results against them. Each pod reads `JOB_COMPLETION_INDEX` to know which shard is its own and prints one JSON line with its result.

Choosing parallelism. I picked **parallelism 4**. The validator is a single-threaded Python loop, so one pod can't use more than one core anyway. I set `requests = limits = 1 CPU` so each pod gets one full core. Then the maths is simple: each node has 2 CPUs so it fits 2 pods, and 2 nodes fit 4 pods in total. 8 shards at 4 at a time is $\lceil 8/4 \rceil = 2$ waves, which is the fastest you can do on 4 CPUs. Parallelism 8 doesn't help. Only 4 pods fit, so the other 4 would just sit Pending, and if I lowered requests to 500m to fit all 8, each pod would get half a core and take twice as long, so it's still two rounds of work. Parallelism 3 would need $\lceil 8/3 \rceil = 3$ waves and leave one CPU idle the whole time. One thing I noticed: minikube's nodes actually show 6 CPUs each, because the Docker driver doesn't limit what kubelet reports. So on my setup the 4 CPU budget is enforced by the Job's own requests and parallelism, not by the nodes.

Manifest. `completionMode: Indexed`, `completions: 8`, `parallelism: 4`, `restartPolicy: Never` (a failed pod keeps its logs and gets retried as a new pod), `backoffLimit: 4`, CPU requests equal to limits, and the Downward API fields `metadata.name` and `spec.nodeName` passed in as env vars.

Proof it ran 4 at once. Peak snapshot from `kubectl get pods -o wide`:

NAME	READY	STATUS	AGE	IP	NODE
shard-validator-0-npkjm	1/1	Running	2s	10.244.1.5	minikube-m02
shard-validator-1-zqc2j	1/1	Running	2s	10.244.1.6	minikube-m02
shard-validator-2-qsf7h	1/1	Running	2s	10.244.0.7	minikube
shard-validator-3-tr6h6	1/1	Running	2s	10.244.0.8	minikube

All 4 running at the same time, 2 on each node. The Job finished 8/8 in 46 s with `Completed Indexes: 0-7` and nothing failed, and the container start and finish times also give a max overlap of 4. Validating a shard only takes about 1 ms, so I made each pod wait 20 s after printing its result, otherwise they'd finish before `kubectl` could even catch them running. That wait doesn't change how many run at once, parallelism still controls that.

shard	0	1	2	3	4	5	6	7	total
invalid rows	25	23	9	10	10	25	15	13	130

All 8 match the expected counts, including the split between missing fields and bad emails.

Why the API and not a shared volume. I got each result with `GET /api/v1/namespaces/default/pods/<pod>/log` instead of making the pods write to a shared volume. A shared volume wouldn't even work on my cluster. minikube's default storage-provisioner makes `hostPath` volumes, which are just a folder on one node's disk, and the minikube docs say it doesn't support multi-node clusters. My pods ran 4 on minikube and 4 on

minikube-m02, so each half would have written to a different node's folder and I'd only see half the results from one place. Logs don't have this problem because each node's kubelet keeps its own pods' logs and the API server fetches them for me, so I can read all 8 from one place no matter where they ran.

If it was 3 nodes with 2 CPUs each. I'd keep it at 4. With 6 CPUs I could run 6 pods at once, but 8 shards with 6 at a time is still $\lceil 8/6 \rceil = 2$ waves (6 and then 2). So the Job takes the same time as with 4, and during the second wave 4 of the 6 CPUs just sit idle. With 4 I get the same finish time and 2 CPUs stay free for other stuff. To actually finish in one wave I'd need 8 CPUs.

Q4: Deployment, self-healing and rolling update

The Deployment has 2 replicas, CPU requests 250m and limits 500m, readiness and liveness probes on `/healthz`, and `maxSurge: 1` with `maxUnavailable: 0`. A NodePort Service on 30080 finds the pods using the `app: spam-api` label. The two replicas ended up on different nodes.

Self-healing

I deleted `spam-api-6db67c8f49-9wq7x`. 2 seconds later there was already a new pod, `spam-api-6db67c8f49-qlwxf`, on `minikube`, Running but 0/1 ready. By 12 s it was 1/1 and the ReplicaSet showed 2/2/2 again. The events show `SuccessfulCreate` from `replicaset-controller`.

This is the ReplicaSet controller's job. My Deployment creates a ReplicaSet with `replicas: 2`, and that controller keeps comparing how many pods match its selector (`app=spam-api` plus the pod-template-hash) with how many it wants. When I deleted `9wq7x` the count dropped to 1, so it made `qlwxf` from the same pod template. The new pod got a new name and IP, but I didn't have to touch the Service because it finds pods by label. It also only started getting traffic after its readinessProbe passed.

Rolling update

For v2 I changed `/healthz` to also show `served_by` (the pod name) and `node`, and built it as `spam-api:v2`. During each rollout a script hit the Service every 0.2 s:

Rollout	old pods had preStop?	failed
v1 → v2 (revision 2)	no	2 / 362
v2 restart (revision 3)	yes, <code>sleep 5</code>	0 / 358

`kubectl rollout status` said success both times, and `rollout history` shows all 3 revisions with their change-causes. The old ReplicaSets stay at 0 so I can roll back. The first rollout had 2 failed requests, both `HTTP=000`, and they happened exactly when each old v1 pod was being shut down. The v1 pod stopped as soon as it got `SIGTERM`, but kube-proxy was still sending it traffic for a moment. So in v2 I added a 5 s `preStop` sleep, which keeps the old pod serving while it gets removed from the Service. In the next rollout the pods being shut down were v2 pods with the `preStop`, and nothing failed.

If this was a Job instead. There wouldn't be a rolling update at all. You can't change a Job's pod template after creating it, so to ship v2 I'd have to delete the Job and make a new one, and the API would be down in between. Also a Job is meant to finish, but `uvicorn` never exits on its own, so the Job would never complete. And whenever a pod exits it counts towards completions or against `backoffLimit`, instead of being replaced to keep 2 replicas running behind the Service.

Deployment vs Job

A Deployment is about keeping something running forever. It keeps N copies of a server up, replaces any that die, and updates versions slowly so the Service always has ready pods. That fits an API that has to answer requests at any time. A

Job is about finishing. It runs until N completions succeed, retries failures up to `backoffLimit`, then stops and keeps the pods around for their logs. That fits Q3, where each shard only needs to be checked once and being done actually means something. If I swapped them, both would break. The API as a Job would try to finish, and the validator as a Deployment (which only allows `restartPolicy: Always`) would keep restarting finished pods and checking the same shards forever.

Where to find the evidence

What it shows	File(s)
Q1 both images build and serve <code>/predict</code> , <code>/healthz</code>	<code>q1_api_verification.txt</code>
Q1 sizes and % reduction	<code>q1_image_sizes.txt</code>
Q1 what the builder left behind	<code>q1_layer_analysis.txt</code>
Q2 cache HIT/MISS with correct labels	<code>q2_cache_headers.txt</code>
Q2 measured speedup	<code>q2_cache_benchmark.txt</code>
Q2 service-name networking	<code>docker-compose.yml</code>
Q3 manifest	<code>k8s/q3-indexed-job.yaml</code>
Q3 parallelism reached	<code>q3_pods_watch_peak.txt</code> , <code>q3_pods_watch.txt</code>
Q3 results through the API	<code>q3_results.txt</code> , <code>q3_job_status.txt</code>
Q4 manifests applied	<code>k8s/q4_pods_initial.txt</code>
Q4 self-healing	<code>q4_self_healing.txt</code>
Q4 rolling update	<code>q4_rollout_*</code> , <code>q4_restart_*</code>

All paths are inside `evidence/` unless shown otherwise. The steps to set up and run everything are in the repo's `README.md`.